Klausur unter Abiturbedingungen · Block 1 (ohne Zusatzthema)

Informatik – Leistungskurs (erhöhtes Anforderungsniveau)

Der Ladepark

Name	
Kurs	Informatik LK, Kurs _____ Lehrkraft: _____
Datum	
Bearbeitungszeit	180 Minuten (einschließlich Einlesezeit)
Hilfsmittel	Ergänzende Hinweise zum Kerncurriculum Informatik (Neufassung 2025): die als Hilfsmittel zugelassenen Abschnitte (u. a. Zeichenketten, Reihungen, Operationen für dynamische Reihungen, Stapel, Schlangen und Binärbäume, Struktogramm- und Klassendiagramm-Notation) Weitere Hilfsmittel sind nicht zugelassen.
Umfang	60 BE · Aufgabe mit den Teilaufgaben a) bis n) · Material (Abbildungen 1 bis 11)

Hinweise

- Bearbeiten Sie alle Teilaufgaben handschriftlich auf dem ausgegebenen Papier. Die Tabellen in den Abbildungen 6 und 9 füllen Sie direkt im Material aus; geben Sie das Material mit ab.
- Implementierungen erfolgen in Java. Die Klassen DynArray, Queue, Stack und BinTree der Bibliothek „Niedersachsen“ sind nicht generisch: Gelesene Inhalte müssen gecastet werden, z. B. `Ladesaeule s = (Ladesaeule) saeulen.getItem(i);`.
- Mögliche Laufzeitfehler (z. B. Entnehmen aus einer leeren Schlange, Zugriff auf einen nicht existierenden Index, Zugriff auf das Wurzelelement eines leeren Baums) müssen durch explizite Abfragen ausgeschlossen werden.
- Struktogramme sind nach den Vorgaben der Ergänzenden Hinweise sprachunabhängig zu notieren.
- Zeitplanung: Rechnen Sie mit etwa 2,5 Minuten je BE; planen Sie Zeit zum Durchlesen und Kontrollieren ein.

Bewertung

Teilaufgabe	a)	b)	c)	d)	e)	f)	g)	h)	i)	j)	k)	l)	m)	n)	Σ
BE	4	3	6	7	6	5	4	4	4	3	3	2	4	5	60
erreicht															

Notenpunkte: _____ Datum, Unterschrift: _____

Aufgabe

Ein Ladepark an einer Autobahnraststätte bietet Ladesäulen für Elektrofahrzeuge mit unterschiedlicher Ladeleistung an. Eine Verwaltungssoftware verwaltet die Ladesäulen, die wartenden Fahrzeuge, die abgeschlossenen Ladevorgänge des Tages, die stündlich geladene Energie, einen Pufferspeicher und die freigeschalteten Ladekarten. Die Modellierung ist in Form eines Klassendiagramms in Abbildung 1 im Material dargestellt. Jedes Fahrzeug ist über sein Kennzeichen eindeutig identifizierbar.

Eine Ladesäule wird durch ein Objekt der Klasse Ladesaeule repräsentiert. Das Attribut leistung gibt die maximale Ladeleistung in kW an. Ist die Säule frei, hat das Attribut aktuell den Wert null. Der Konstruktor Ladesaeule(nummer: Ganzzahl) erzeugt eine Säule mit der Standardleistung 11 kW.

a) Erläutern Sie am Beispiel der Klasse Ladesaeule den grundsätzlichen Aufbau einer Klasse. Gehen Sie dabei auch auf die beiden Konstruktoren und auf die Sichtbarkeitsangaben (+ und -) ein.

[4 BE]

Der Ladepark wird um Schnellladesäulen erweitert. Eine Schnellladesäule ist eine Ladesäule, die zusätzlich ihren Steckertyp (z. B. "CCS") speichert. Für das Laden an einer Schnellladesäule wird auf den Preis pro kWh einer gewöhnlichen Ladesäule ein säulenspezifischer Zuschlag aufgeschlagen.

b) Ergänzen Sie das Klassendiagramm um eine Klasse Schnellladesaeule. Beschränken Sie sich dabei auf die Elemente, die gegenüber der Klasse Ladesaeule neu sind oder sich ändern.

[3 BE]

Alle Ladesäulen des Ladeparks werden in der dynamischen Reihung saeulen verwaltet.

c) Die Operation besteFreieSaeule(): Ladesaeule der Klasse Ladepark gibt die freie Ladesäule mit der größten Ladeleistung zurück. Haben mehrere freie Säulen diese Ladeleistung, wird diejenige mit dem kleinsten Index zurückgegeben. Ist keine Säule frei oder enthält saeulen keine Säule, wird null zurückgegeben.

Implementieren Sie die Operation besteFreieSaeule in einer im Unterricht verwendeten Programmiersprache.

[6 BE]

Ist keine Säule frei, wird für das ankommende Fahrzeug ein Objekt der Klasse Ladevorgang mit der gewünschten Energie (in kWh) und der voraussichtlichen Ladedauer (in Minuten) erzeugt und in die Schlange warteschlange eingereiht. Abbildung 2 zeigt ein Beispiel.

d) Die Operation wartezeit(kennzeichen: Zeichenkette): Ganzzahl der Klasse Ladepark gibt die Summe der Ladedauern aller Ladevorgänge zurück, die in warteschlange vor dem Ladevorgang mit dem übergebenen Kennzeichen stehen. Ist kein Ladevorgang mit diesem Kennzeichen in der Schlange enthalten, wird −1 zurückgegeben. Für die Schlange aus Abbildung 2 liefert der Aufruf wartezeit("HB-AQ 300") den Wert 70. Nach Ausführung der Operation muss warteschlange unverändert vorliegen.

Implementieren Sie die Operation wartezeit in einer im Unterricht verwendeten Programmiersprache.

[7 BE]

In der zweidimensionalen Reihung energie ist für jede Ladesäule und jede Stunde des Tages die geladene Energie in kWh gespeichert. Der Zeilenindex entspricht dem Index der Säule in saeulen, der Spaltenindex der Stunde (0 bis 23). Abbildung 3 zeigt einen Auszug. Sie können davon ausgehen, dass der Ladepark mindestens eine Säule besitzt.

e) Die Operation spitzenstunde(): Ganzzahl der Klasse Ladepark gibt die Stunde zurück, in der im gesamten Ladepark am meisten Energie geladen wurde. Gibt es mehrere solche Stunden, wird die früheste zurückgegeben. Für den Auszug in Abbildung 3 wäre dies die Stunde 17.

Entwickeln Sie ein Struktogramm für die Operation spitzenstunde.

[6 BE]

Die abgeschlossenen Ladevorgänge des Tages werden für die Abrechnung in der dynamischen Reihung protokoll gespeichert. Die Operation sortiereProtokoll(), die in Abbildung 4 als Struktogramm gegeben ist, ordnet sie nach der geladenen Energie.

f) Dokumentieren Sie in der Tabelle in Abbildung 6 die Verarbeitung der Beispieldaten aus Abbildung 5 durch die Operation sortiereProtokoll. **[5 BE]**

g) Analysieren Sie die Funktionsweise der Operation sortiereProtokoll. Erläutern Sie dabei die Bedeutung der Abfrage pos = 0 und begründen Sie, dass protokoll nach Ablauf der Operation absteigend nach der Energie sortiert ist. **[4 BE]**

h) Die Laufzeit der Operation sortiereProtokoll wird wesentlich durch die Anzahl der Vertauschungen bestimmt, d. h. durch die Anzahl der Ausführungen des wahr-Zweigs der inneren Verzweigung.

Bestimmen Sie die Anzahl der Vertauschungen für ein protokoll mit n Ladevorgängen, das bereits absteigend bzw. streng aufsteigend nach der Energie sortiert ist. Schätzen Sie ab, wie sich die Laufzeit im ungünstigsten Fall verändert, wenn sich die Anzahl der Ladevorgänge verdoppelt. **[4 BE]**

Damit der Netzanschluss nicht überlastet wird, besitzt der Ladepark einen Pufferspeicher (eine stationäre Batterie). Aus dem Netz werden pro Stunde höchstens netzleistung kWh bezogen. Ist der Bedarf einer Stunde kleiner, wird der Rest in den Pufferspeicher geladen, jedoch höchstens bis zu dessen Kapazität kapazitaet. Ist der Bedarf größer, wird die Differenz dem Pufferspeicher entnommen. Reicht der Pufferspeicher dafür nicht aus, liegt eine Überlast vor.

Die rekursive Operation ueberlast (Abbildung 7) gibt für eine Prognose bedarf, die den Bedarf in kWh für die kommenden Stunden 0, 1, 2, … enthält, die erste Stunde mit Überlast zurück bzw. −1, wenn keine Überlast auftritt. Der Parameter puffer gibt den Füllstand des Pufferspeichers in kWh zu Beginn der Stunde h an. Im Folgenden gilt netzleistung = 100 und kapazitaet = 120.

i) Dokumentieren Sie den Ablauf des Aufrufs ueberlast(bedarf, 0, 60) für die Prognose aus Abbildung 8 in der Tabelle in Abbildung 9. **[4 BE]**

j) Erläutern Sie das Prinzip der Rekursion anhand der Operation ueberlast. **[3 BE]**

k) Begründen Sie, dass die Rekursionstiefe des letzten Aufrufs genau dann der Anzahl der Elemente von bedarf entspricht, wenn die Operation beim Erstaufruf ueberlast(bedarf, 0, p) den Wert −1 zurückgibt. **[3 BE]**

Zum Laden wird eine freigeschaltete Ladekarte benötigt. Die Nummern der freigeschalteten Ladekarten werden im binären Suchbaum karten gespeichert. Für jeden Knoten gilt: Alle Nummern im linken Teilbaum sind kleiner, alle Nummern im rechten Teilbaum größer als die Nummer des Knotens.

l) Zeichnen Sie den binären Suchbaum, der entsteht, wenn die Kartennummern 518, 274, 730, 133, 402, 615, 911, 350, 461, 688 in dieser Reihenfolge in einen leeren binären Suchbaum eingefügt werden. **[2 BE]**

Für Auswertungen wird die rekursive Operation anzahlImBereich verwendet, die in Abbildung 10 als Struktogramm gegeben ist.

m) Bestimmen Sie die Rückgabe des Aufrufs anzahlImBereich(karten, 300, 450) für den Baum aus Teilaufgabe l) und geben Sie die Inhalte der Wurzeln aller nicht leeren Teilbäume an, für die die Operation dabei aufgerufen wird.

Begründen Sie, dass die Operation die richtige Anzahl liefert, obwohl im Fall x < von der linke Teilbaum nicht untersucht wird. **[4 BE]**

Die Operation entferneVorgaenge(kennzeichen: Zeichenkette): Ganzzahl der Klasse Ladepark soll alle Ladevorgänge mit dem übergebenen Kennzeichen aus protokoll entfernen und deren Anzahl zurückgeben. Die Reihenfolge der übrigen Ladevorgänge soll erhalten bleiben. Die Implementierung in Abbildung 11 enthält drei Fehler.

n) Geben Sie die drei Fehler an, erläutern Sie jeweils deren Auswirkung und korrigieren Sie die Fehler. **[5 BE]**

Material

zu den Aufgabenteilen a) bis e) und n)

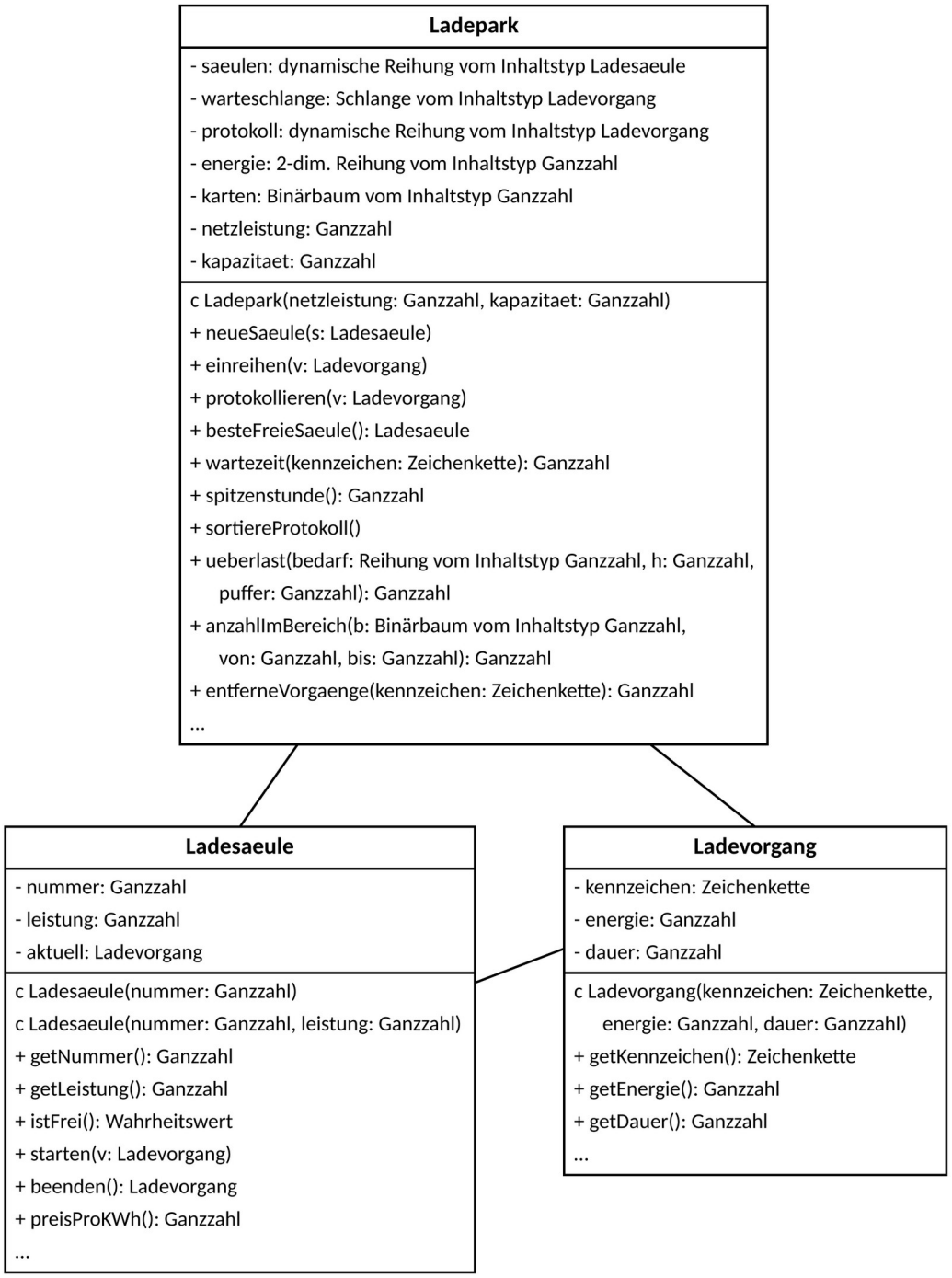


Abbildung 1: Klassendiagramm der Verwaltungssoftware des Ladeparks (Ausschnitt)

	vorne			hinten
Kennzeichen	DEL-LP 42	OL-EV 7	HB-AQ 300	BRA-K 19
Energie (kWh)	30	18	52	12
Ladedauer (min)	45	25	60	20

Abbildung 2: Beispiel für die Schlange warteschlange vom Inhaltstyp Ladevorgang

Säule \ Stunde	15	16	17	18	19	20
0 (11 kW)	9	11	11	10	6	0
1 (50 kW)	40	45	48	50	32	21
2 (22 kW)	0	22	22	11	22	22
3 (150 kW)	80	95	120	110	120	60

Abbildung 3: Auszug aus der Reihung energie (Zeilen 0 bis 3, Spalten 15 bis 20; Werte in kWh). Die übrigen Spalten 0 bis 14 und 21 bis 23 sind nicht dargestellt.

zu den Aufgabenteilen f) bis h)

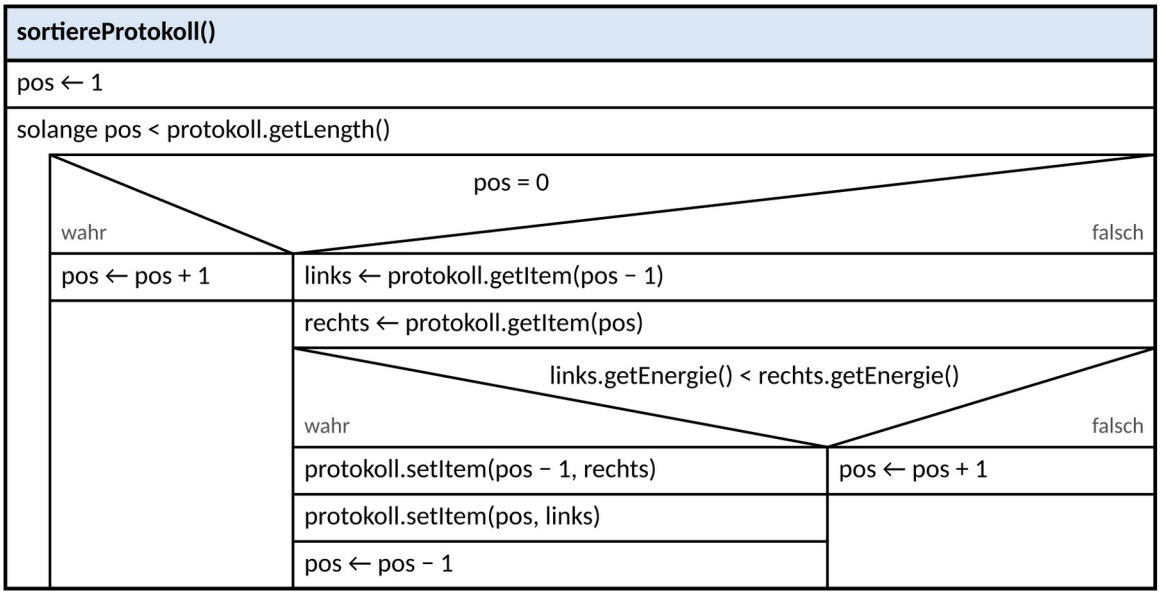


Abbildung 4: Struktogramm der Operation sortiereProtokoll

Index	0	1	2	3	4
Kennzeichen	OL-EV 7	HB-AQ 300	DEL-LP 42	WST-M 5	BRA-K 19
Energie (kWh)	18	42	25	42	9

Abbildung 5: Beispieldaten zur dynamischen Reihung protokoll vom Inhaltstyp Ladevorgang

pos zu Beginn des Durchlaufs	[0]	[1]	[2]	[3]	[4]
vor der Schleife: 1	18	42	25	42	9

Wert von pos nach Ende der Schleife: _____

Abbildung 6: Tracetabelle zur Vervollständigung. Tragen Sie die Zeilen an folgenden Stellen des Struktogramms ein: die erste Zeile (Startwerte) nach der Anweisung pos ← 1 vor der Schleife (vorgegeben), danach je eine Zeile am Ende jedes Schleifendurchlaufs, d. h. nach der Verzweigung pos = 0 als letzter Schritt im Schleifenrumpf. Notieren Sie jeweils den Wert von pos zu Beginn des Durchlaufs und die Energiewerte in protokoll am Ende des Durchlaufs.

zu den Aufgabenteilen i) bis k)

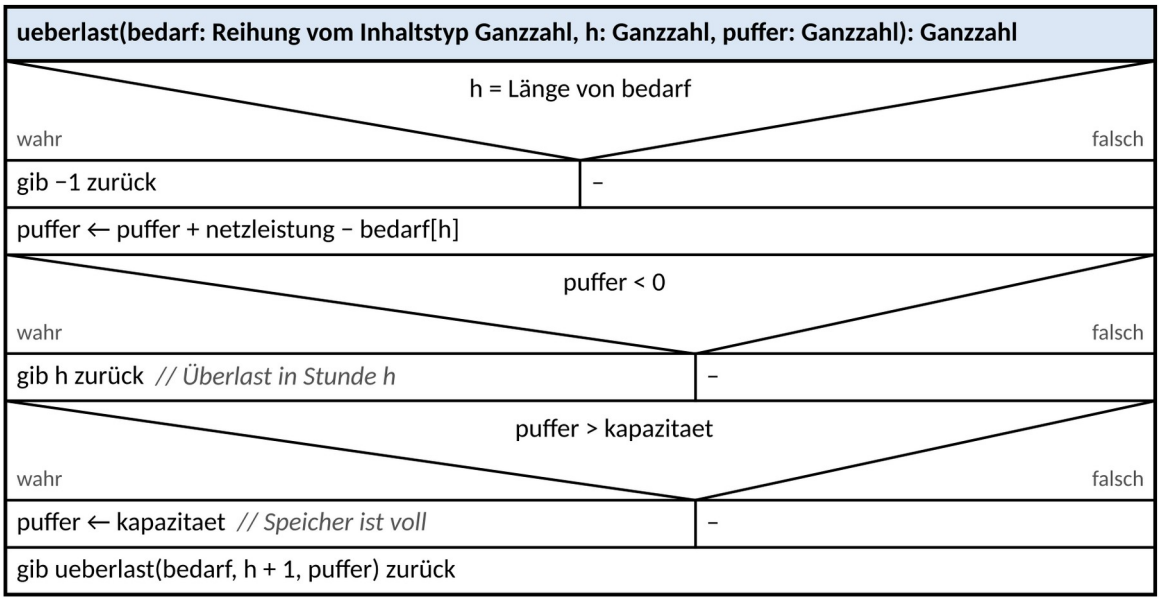


Abbildung 7: Struktogramm der rekursiven Operation ueberlast

Index	0	1	2	3	4	5	6	7
bedarf (kWh)	20	130	160	90	150	140	60	180

Abbildung 8: Prognose bedarf für die kommenden acht Stunden (netzleistung = 100, kapazitaet = 120)

Rekursionstiefe *)	h	puffer beim Aufruf	bedarf[h]	puffer nach der Berechnung (ggf. begrenzt)	Rückgabe
0 (Erstaufruf)	0	60			

*Abbildung 9: Tabelle zur Dokumentation des Aufrufs ueberlast(bedarf, 0, 60). *) Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1; der Erstaufruf hat die Rekursionstiefe 0. Nicht benötigte Zeilen bleiben leer.*

zu Aufgabenteil m)

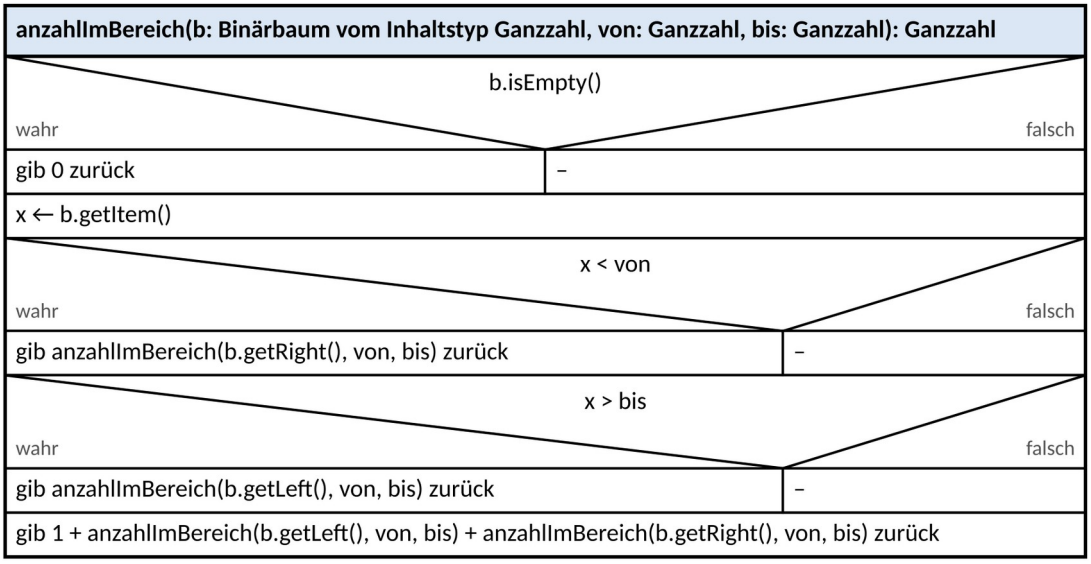


Abbildung 10: Struktogramm der rekursiven Operation anzahlImBereich

zu Aufgabenteil n)

```
 1  public int entferneVorgaenge(String kennzeichen) {
 2      int anzahl = 0;
 3      for (int i = 0; i <= protokoll.getLength(); i++) {
 4          Ladevorgang v = (Ladevorgang) protokoll.getItem(i);
 5          if (v.getKennzeichen() == kennzeichen) {
 6              protokoll.delete(i);
 7              anzahl++;
 8          }
 9      }
10      return anzahl;
11  }
```

Abbildung 11: Fehlerhafte Implementierung der Operation entferneVorgaenge (mit Zeilennummern)